UNIVERSITÉ CAEN NORMANDIE

UFR des Sciences

Université de Caen Normandie
Département mathématiques-informatique
2024–2025
Bruno Zanuttini

L3 informatique

# TP. Extraction de connaissances

Unité YSINL5D2 : Aide à la décision et intelligence artificielle

---

L'objectif de ce TP est d'écrire des classes permettant de lancer des processus d'extraction de connaissances. Toutes les classes créées pour ce faire devront l'être dans un *package* Java nommé `datamining`.

Il s'agit essentiellement d'implémenter des algorithmes d'extraction de motifs (*itemsets*) et de règles d'association. Pour cela, on écrira des classes (par exemple `Apriori`) dont chaque instance sera dédiée à l'extraction dans une base de données. Ainsi, la méthode d'extraction n'aura plus à prendre la base en argument (c'est l'attribut de la classe qui sera utilisé) ; elle aura seulement à prendre la fréquence minimale désirée (et la confiance pour l'extraction des règles).

Cet énoncé suppose que les classes `Variable` et `BooleanVariable` du *package* `modelling` sont implémentées (cf. 1er TP).

## Tests fournis

Des tests unitaires sont fournis dans le package `dataminingtests`. Un exemple complet d'utilisation de ces tests est :

```
import dataminingtests.AbstractAssociationRuleMinerTests;
import dataminingtests.AbstractItemsetMinerTests;
import dataminingtests.AprioriTests;
import dataminingtests.BruteForceAssociationRuleMinerTests;
[...]
boolean ok = true;
ok = ok && AbstractItemsetMinerTests.testFrequency();
ok = ok && AprioriTests.testFrequentSingletons();
ok = ok && AprioriTests.testCombine();
ok = ok && AprioriTests.testAllSubsetsFrequent();
ok = ok && AprioriTests.testExtract();
ok = ok && AbstractAssociationRuleMinerTests.testFrequency();
ok = ok && AbstractAssociationRuleMinerTests.testConfidence();
ok = ok &&
    BruteForceAssociationRuleMinerTests.testAllCandidatePremises();
ok = ok && BruteForceAssociationRuleMinerTests.testExtract();
System.out.println(ok ? "All tests OK" : "At least one test KO");
```

Il est aussi possible de tester ses propres algorithmes d'extraction de motifs ou de règles en appelant :

```
ok = ok && new ItemsetMinerTests(
    database -> new MonExtracteur(database...)
).testExtract();
ok = ok && new AssociationRuleMinerTests(
    database -> new MonExtracteur(database...)
).testExtract();
```

# 1 Bases de données et motifs

**Note** De façon générale, on ne considérera que des bases de données décrites par des attributs booléens (donc des bases transactionnelles). On représentera ainsi les *items* par des instances de la classe `BooleanVariable`, et les transactions et les motifs par des ensembles d'*items* (`Set<BooleanVariable>`). Pour certaines méthodes toutefois, il sera utile que les ensembles d'*items* soient triés ; pour cela on utilisera le type `SortedSet<BooleanVariable>` (`SortedSet<...>` est une interface, implémentée notamment par `TreeSet<...>` ; les deux appartiennent au *package* `java.util`). Un exemple typique d'utilisation est :

```
SortedSet<BooleanVariable> items = new TreeSet<>(myComparator);
items.add(...)
```

Les instructions plus précises sur ces aspects sont données au fil de l'énoncé.

**Exercice 1.** *Créer une classe nommée* `BooleanDatabase`*, pour représenter des bases de données transactionnelles. Le constructeur devra prendre en argument l'ensemble des* items *(*`Set<BooleanVariable>`*) et construire une base vide. Une méthode* `void add(Set<BooleanVariable>)` *devra permettre d'ajouter une transaction, et la classe devra fournir des accesseurs* `Set<BooleanVariable> getItems()` *et* `List<Set<BooleanVariable>> getTransactions()`*.*

**Exercice 2.** *Créer une classe nommée* `Itemset`*, dont le constructeur prend en arguments un ensemble d'*items *(*`Set<BooleanVariable>`*) et une fréquence (de type* `float`*, entendue entre 0,0 pour un motif non supporté, et 1,0 pour un motif supporté par toutes les transactions). Munir la classe d'accesseurs* `Set<BooleanVariable> getItems()` *et* `float getFrequency()`*.*

# 2 Extraction d'*itemsets* fréquents

**Exercice 3.** *Créer une interface nommée* `ItemsetMiner`*, déclarant une méthode* `BooleanDatabase getDatabase()`*, ainsi qu'une méthode* `Set<Itemset> extract(float)` *prenant en argument une fréquence minimale, et retournant l'ensemble des* itemsets *(non vides) ayant au moins cette fréquence dans la base.*

**Exercice 4.** *Créer une classe abstraite nommée* `AbstractItemsetMiner` *implémentant l'interface* `ItemsetMiner`*, munie d'un constructeur prenant en argument une base (de type* `BooleanDatabase`*), d'un accesseur pour cette base, et d'une méthode* `float frequency(Set<BooleanVariable>)` *retournant la fréquence d'un ensemble donné d'*items *dans la base.*

**Exercice 5.** *Ajouter à la classe* `AbstractItemsetMiner` *un attribut statique* `COMPARATOR` *permettant de comparer les* items *(pour les instances de* `SortedSet<BooleanVariable>`*), en déclarant par exemple :*

```
public static final Comparator<BooleanVariable> COMPARATOR =
    (var1, var2) -> var1.getName().compareTo(var2.getName());
```

**Exercice 6.** *Écrire une classe nommée* `Apriori`*, étendant la classe* `AbstractItemsetMiner`*, et munie d'un constructeur prenant en argument une base (de type* `BooleanDatabase`*) ; cette classe représentera des extracteurs fonctionnant sur le principe de l'algorithme « apriori ».*

Il s'agit maintenant d'implémenter l'algorithme d'extraction à proprement parler.

**Exercice 7.** *Ajouter à la classe* `Apriori` *une méthode* `Set<Itemset> frequentSingletons(float)` *prenant en argument une fréquence, et retournant l'ensemble de tous les* itemsets *singletons (avec leur fréquence) dont la fréquence dans la base est au moins égale à celle donnée.*

**Exercice 8.** *Ajouter à la classe* `Apriori` *une méthode statique* `combine`*, prenant en arguments deux ensembles (triés) d'*items *(de type* `SortedSet<BooleanVariable>`*) et retournant un ensemble (trié) d'*items *(également de type* `SortedSet<BooleanVariable>`*) obtenu en les combinant, à condition que*

- *les deux ensembles aient la même taille $k$,*
- *les deux ensembles aient les mêmes $k-1$ premiers* items*,*
- *les deux ensembles aient des $k^{es}$* items *différents.*

*Dans tous les autres cas, la méthode devra renvoyer* `null`*.*

**Exercice 9.** *Ajouter à la classe* `Apriori` *une méthode statique* `allSubsetsFrequent` *prenant en arguments un ensemble d'*items *de type* `Set<BooleanVariable>` *(en pratique, de taille $k$) et une collection d'ensembles d'*items *de type* `Collection<SortedSet<BooleanVariable>>` *(en pratique, les fréquents de*

*taille $k-1$), et retournant* `true` *si tous les sous-ensembles obtenus en supprimant exactement un élément de l'ensemble d'*items *sont contenus dans la collection (*`false` *sinon). L'idée derrière cette méthode est que la propriété d'antimonotonie de la fréquence assure alors que les sous-ensembles propres de l'ensemble donné sont également fréquents.*

**Exercice 10.** *En utilisant les méthodes précédentes, définir dans la classe* `Apriori` *la méthode* `extract` *déclarée par l'interface* `ItemsetMiner`*. Pour créer un ensemble trié (vide), on pourra utiliser l'expression*

```
new TreeSet<>(AbstractItemsetMiner.COMPARATOR)
```

Indication : *Il sera utile, en plus du résultat de type* `Set<Itemset>` *construit au fur et à mesure, de maintenir une variable de type* `List<SortedSet<BooleanVariable>>` *pour stocker les ensembles d'*items *fréquents découverts au niveau $k$ lorsque l'on calcule les fréquents du niveau $k+1$.*

# 3 Extraction de règles fréquentes et précises

**Exercice 11.** *Créer une classe nommée* `AssociationRule`*, dont le constructeur prend en arguments, dans cet ordre, une prémisse, une conclusion (les deux de type* `Set<BooleanVariable>`*), ainsi qu'une fréquence et une confiance (de type* `float`*), et qui fournit un accesseur à chacune de ces propriétés.*

**Exercice 12.** *Créer une interface nommée* `AssociationRuleMiner`*, qui déclare une méthode nommée* `getDatabase` *retournant une instance de* `BooleanDatabase`*, et une méthode nommée* `extract`*, qui prend en arguments une fréquence et une confiance minimales (de type* `float`*) et qui retourne l'ensemble (de type* `Set<AssociationRule>`*) des règles d'association de fréquence et confiance supérieures aux seuils donnés.*

**Exercice 13.** *Créer une classe abstraite nommée* `AbstractAssociationRuleMiner`*, implémentant l'interface* `AssociationRuleMiner`*, dont le constructeur prend en argument une base de données (de type* `BooleanDatabase`*), et qui définit l'accesseur à cette base.*

**Exercice 14.** *Ajouter à la classe* `AbstractAssociationRuleMiner` *une méthode statique* `frequency` *qui prend en arguments, dans cet ordre, un ensemble d'*items *(de type* `Set<BooleanVariable>`*) et un ensemble d'*itemsets *(de type* `Set<Itemset>`*), et qui retourne la fréquence de l'ensemble d'*items *telle qu'elle est stockée dans l'ensemble d'*itemsets*. Pour ce qui est de l'utilisation de cette méthode dans la suite, il s'agira de retrouver la fréquence d'un ensemble d'*items *candidat à la création d'une règle d'association, dans l'ensemble préalablement calculé des* itemsets *fréquents. Si l'ensemble d'*items *n'apparaît pas dans l'ensemble d'*itemsets*, on pourra jeter une exception de type* `IllegalArgumentException`*, ou retourner* `null` *(dans l'utilisation que l'on fera de cette méthode, cette situation ne devrait pas arriver).*

**Exercice 15.** *Ajouter à la classe* `AbstractAssociationRuleMiner` *une méthode statique* `confidence` *qui prend en arguments, dans cet ordre, deux ensembles d'*items *(de type* `Set<BooleanVariable>`*) représentant respectivement une prémisse et une conclusion (candidates), ainsi qu'un ensemble d'*itemsets *(de type* `Set<Itemset>`*), et qui retourne une valeur de type* `float` *représentant la confiance de la règle d'association de prémisse et conclusion données, les fréquences d'*itemsets *nécessaires aux calculs étant données dans l'ensemble en argument.*

**Exercice 16.** *Créer une classe nommée* `BruteForceAssociationRuleMiner`*, qui hérite de la classe* `AbstractAssociationRuleMiner`*, et munie d'un constructeur prenant en argument une base de données (de type* `BooleanDatabase`*).*

Il s'agit maintenant implémenter un algorithme *brute-force* d'énumération des règles d'association fréquentes et précises à partir de l'ensemble des motifs fréquents.

**Exercice 17.** *Ajouter à la classe* `BruteForceAssociationRuleMiner` *une méthode statique nommée* `allCandidatePremises`*, qui prend en argument un ensemble d'*items *(de type* `Set<BooleanVariable>`*), et retourne l'ensemble de tous ses sous-ensembles (*`Set<Set<BooleanVariable>>`*), à l'exception de l'ensemble vide et de l'ensemble lui-même.*

**Exercice 18.** *En utilisant les méthodes précédentes, définir la méthode* `extract` *déclarée par l'interface* `AssociationRuleMiner`*.*